

RIPHAH INTERNATIONAL COLLEGE HARBANSPURA CAMPUS

A Project of Riphah International University

CS-III: Artificial Intelligence

10 Industry-Ready Final Year Project Proposals

Course: Artificial Intelligence | Course ID: CS-III | Credits: 3 (2-3)

Stack: Python · Streamlit · Scikit-learn · PyTorch · MySQL · NLTK · SpaCy · OpenCV

All projects run on LOCAL SYSTEM only | No cloud deployment required | All tools FREE

Duration per project: 6–8 weeks

Introduction & Guidelines

The following 10 project proposals are designed for students enrolled in CS-III: Artificial Intelligence at Riphah International Colleges. Each project is:

- Built entirely on free, open-source tools — no paid APIs or cloud subscriptions required.
- Designed to run on a student's local machine (Windows/Linux/macOS) or free Kaggle/Google Colab notebooks.
- Scoped for 2 months (6–8 weeks) of part-time student effort.
- Aligned with the course CLOs (CLO1–CLO6) covering search, ML, NLP, knowledge representation, and AI applications.
- Structured for international-level presentation and prize-winning potential at national tech competitions.

Common Technology Stack

Language	Python 3.x (free, open-source)
Web UI	Streamlit (free, localhost deployment)
ML Library	Scikit-learn (classification, regression, clustering)
Deep Learning	PyTorch (free, runs on CPU without GPU)
NLP	NLTK, SpaCy (both free)
Computer Vision	OpenCV (free)
Database	MySQL Community Edition (free, local)
Datasets	Kaggle (free account) or UCI ML Repository
Notebooks	Google Colab Free Tier / Kaggle Notebooks (free GPU)

Project 1 | **AI-Powered Medical Symptom Checker & Disease Prediction System**

Category: Machine Learning + AI Web App | Duration: 6–8 weeks

Abstract

A Streamlit-based intelligent web application that accepts patient-reported symptoms as input and predicts probable diseases using a trained Random Forest and Naive Bayes ensemble classifier. The system recommends precautions and doctor specializations based on predictions. All patient data is stored locally in MySQL. No cloud deployment is required — runs fully on localhost.

Real-World Problem Statement

Patients in rural or under-resourced areas lack quick access to preliminary medical guidance. This system acts as a first-line intelligent assistant to reduce unnecessary hospital visits.

Functional Requirements

- User can select multiple symptoms from a dropdown menu
- System predicts top-3 probable diseases with confidence scores
- Displays recommended precautions and suggested medical specialist
- Patient query history stored in MySQL database
- Admin panel to view statistics (most queried symptoms, predicted diseases)
- Export report as PDF from Streamlit UI
- Input validation and error handling for incomplete symptom sets

Tools & Technologies

Python 3.x • Streamlit • Scikit-learn (Random Forest, Naive Bayes) • MySQL + mysql-connector-python
• Pandas • Matplotlib / Seaborn • ReportLab (PDF export) • Kaggle Disease-Symptom Dataset (free)

Prize Potential: Healthcare AI + real-world impact = strong prize candidate at national computing competitions (ICPC Pakistan, SOFTEC, FAST NUCES).

Project 2 | Fake News Detection System Using NLP & Machine Learning

Category: NLP + Machine Learning | Duration: 6–7 weeks

Abstract

A Streamlit web app that classifies news articles as real or fake using TF-IDF vectorization combined with Logistic Regression and LSTM deep learning models. Users can paste a news article URL or raw text and instantly receive a credibility score with highlighted suspicious keywords. The LIAR or FakeNewsNet dataset (both free on Kaggle) is used for training.

Real-World Problem Statement

Misinformation spreads rapidly through social media in Pakistan and globally. This tool empowers journalists, students, and the public to verify news authenticity instantly without internet dependency.

Functional Requirements

- User can paste raw article text or paste a news URL for scraping
- System classifies news as Real / Fake with confidence percentage
- Highlights high-risk words or phrases contributing to the classification
- Comparison view: Logistic Regression vs LSTM model results
- News history saved in MySQL with timestamps
- Visual statistics dashboard (pie charts, word clouds)
- Downloadable analysis report

Tools & Technologies

Python 3.x • Streamlit • Scikit-learn (TF-IDF, Logistic Regression) • PyTorch (LSTM model) • NLTK / SpaCy • BeautifulSoup4 (URL scraping) • MySQL • WordCloud library • Kaggle LIAR Dataset (free)

Prize Potential: Highly relevant to current global issues — media, journalism, and cybersecurity panels consistently reward this topic.

Project 3 | **Smart Student Performance Predictor & Academic Advisor**

Category: Machine Learning + Education AI | Duration: 6–8 weeks

Abstract

An ML-powered Streamlit application for educational institutions that predicts student final grades and dropout risk using Decision Trees, Random Forest, and Linear Regression. The system provides personalized academic recommendations based on attendance, assignment scores, and quiz results. Data is managed locally in MySQL.

Real-World Problem Statement

Universities struggle to identify at-risk students before it is too late. Early-warning AI systems can help faculty intervene proactively, improving retention and academic outcomes.

Functional Requirements

- Input form for student data: attendance %, quiz scores, assignment marks, mid-term grade
- Predicts final grade range and dropout probability
- Generates personalized advice (e.g., 'Focus on lab sessions', 'Improve quiz performance')
- Admin dashboard: class-wide analytics, risk heatmap
- MySQL storage for all student records
- Batch CSV upload for processing entire class at once
- Performance trend visualization per student over weeks

Tools & Technologies

Python 3.x • Streamlit • Scikit-learn (Decision Tree, Random Forest, Linear Regression) • Pandas • Matplotlib / Plotly • MySQL • OpenPyXL (CSV/Excel import) • Kaggle Student Performance Dataset (free)

Prize Potential: Education technology is a top-priority sector for government and private tech competitions in Pakistan.

Project 4 | Resume Screening & Job Match AI System

Category: NLP + AI Web App | Duration: 7–8 weeks

Abstract

An NLP-driven Streamlit application that automates resume screening for recruiters. The system parses uploaded PDF resumes, extracts skills and experience using SpaCy NER, and matches them against job descriptions using cosine similarity. A ranked shortlist is generated automatically. All data is stored in MySQL with no cloud dependency.

Real-World Problem Statement

HR teams in companies receive hundreds of resumes per job opening. Manual screening is time-consuming and biased. This tool reduces screening time by 80% and provides objective matching.

Functional Requirements

- HR uploads job description (text) and multiple PDF resumes
- System extracts: Name, Skills, Education, Experience from resumes using NLP
- Cosine similarity score computed between each resume and job description
- Ranked shortlist table with match percentage
- Individual resume analysis report with missing skills highlighted
- MySQL storage for all job postings and candidate profiles
- Export shortlist to Excel

Tools & Technologies

Python 3.x • Streamlit • SpaCy (NER + NLP pipeline) • Scikit-learn (TF-IDF, Cosine Similarity) • PyMuPDF / pdfplumber (PDF parsing) • MySQL • Pandas • OpenPyXL

Prize Potential: HR Tech + NLP combination is a proven winner at startup competitions and AI hackathons.

Project 5 | **Real-Time Sentiment Analysis Dashboard for Product Reviews**

Category: NLP + Machine Learning + Visualization | Duration: 5–7 weeks

Abstract

A Streamlit dashboard that performs real-time sentiment analysis (Positive / Negative / Neutral) on product reviews using VADER, TextBlob, and a fine-tuned Logistic Regression model. Users can upload CSV files of reviews or type reviews manually. Results are visualized with interactive charts and stored in MySQL for trend analysis over time.

Real-World Problem Statement

E-commerce businesses and product managers need to understand customer sentiment at scale. Manual reading of thousands of reviews is impractical. This tool provides instant actionable insights.

Functional Requirements

- Manual review input or bulk CSV file upload
- Sentiment classification: Positive, Negative, Neutral with score
- Aspect-based sentiment (extracts opinions about: price, quality, delivery, etc.)
- Interactive charts: pie chart, sentiment over time line graph, word cloud
- Comparison of VADER vs ML model predictions
- MySQL storage for all analyzed reviews with timestamps
- Summary statistics: overall rating, most negative aspects, trend alerts

Tools & Technologies

Python 3.x • Streamlit • NLTK (VADER Sentiment) • TextBlob • Scikit-learn (Logistic Regression, TF-IDF) • Plotly • WordCloud • MySQL • Pandas • Kaggle Amazon Reviews Dataset (free)

Prize Potential: Business intelligence + NLP makes this highly applicable — strong candidate for business/data competitions.

Project 6 | **AI-Based Traffic Sign Recognition System**

Category: Computer Vision + Deep Learning | Duration: 7–8 weeks

Abstract

A PyTorch-based Convolutional Neural Network (CNN) trained on the GTSRB dataset to recognize 43 classes of German traffic signs. A Streamlit interface allows users to upload road images and receive instant classification results. The system also generates a confidence heatmap (Grad-CAM visualization) showing which image regions influenced the prediction.

Real-World Problem Statement

Autonomous driving and driver assistance systems require accurate, real-time traffic sign recognition. This project demonstrates a core component of self-driving AI technology using only free tools.

Functional Requirements

- Upload road image or use webcam capture (via OpenCV)
- CNN classifies traffic sign with top-3 predictions and confidence scores
- Grad-CAM heatmap overlay visualization showing decision regions
- Model training pipeline with accuracy/loss graphs
- Confusion matrix and per-class accuracy in admin view
- MySQL logging of all predictions with timestamps
- Model export and reload functionality

Tools & Technologies

Python 3.x • PyTorch + torchvision • Streamlit • OpenCV • Matplotlib • Scikit-learn (evaluation metrics) • MySQL • Grad-CAM (pytorch-grad-cam library) • Kaggle GTSRB Dataset (free)

Prize Potential: Computer Vision + Autonomous Driving is the most glamorous AI research area — guaranteed attention at any tech expo.

Project 7 | Intelligent Chatbot for University FAQ & Course Advisor

Category: NLP + Knowledge Representation + AI Web App | Duration: 6–7 weeks

Abstract

A rule-based and ML-hybrid chatbot built with NLTK and Streamlit that answers university-related queries such as fee structure, course registration, exam schedules, and department contacts. The knowledge base is stored as a MySQL-backed ontology. An intent classification model (Naive Bayes) handles natural variations in user questions.

Real-World Problem Statement

University help desks are overwhelmed with repetitive student queries. A locally-running intelligent chatbot can handle 80% of FAQs without human intervention, saving time for both students and staff.

Functional Requirements

- Conversational interface with chat history in Streamlit
- Intent recognition for 20+ university-related topics
- Retrieves answers from MySQL knowledge base (ontology-structured)
- Admin panel to add/edit FAQ entries without coding
- Handles spelling mistakes and informal Urdu-English (Romanized) input
- Conversation logging for analysis in MySQL
- Fallback to 'Contact admin' for unrecognized queries

Tools & Technologies

Python 3.x • Streamlit • NLTK (tokenization, stemming, intent classification) • Scikit-learn (Naive Bayes) • MySQL • Pandas • FuzzyWuzzy (typo handling) • JSON knowledge base

Prize Potential: Highly practical for real institutional deployment — judges love projects with immediate real-world utility.

Project 8 | Crop Disease Detection System Using Deep Learning

Category: Computer Vision + Deep Learning + AI Web App | Duration: 7–8 weeks

Abstract

A PyTorch CNN-based system trained on the PlantVillage dataset (Kaggle, free) to identify 38 classes of crop diseases from leaf images. The Streamlit interface allows farmers or agricultural officers to upload plant leaf photos and receive instant disease diagnosis with treatment recommendations. All data is stored in MySQL locally.

Real-World Problem Statement

Pakistani agriculture suffers billions in losses annually due to undetected or misidentified crop diseases. An AI-powered early detection tool accessible on local devices can dramatically reduce crop losses.

Functional Requirements

- Upload plant leaf image or capture via webcam
- CNN identifies disease type with confidence score from 38 classes
- Displays disease description, symptoms, and recommended treatment
- Supports 5+ major crops (wheat, rice, tomato, potato, corn)
- MySQL database of disease records, treatments, and pesticide info
- Admin can add new disease entries to the knowledge base
- Report generation per diagnosis for farmer record-keeping

Tools & Technologies

Python 3.x • PyTorch + torchvision (ResNet18 transfer learning) • Streamlit • OpenCV • Pillow • MySQL • Pandas • Matplotlib • Kaggle PlantVillage Dataset (free, 54,000+ images)

Prize Potential: Agriculture AI in Pakistan context = massive impact score. National innovation competitions and USAID-backed tech events actively seek this type of project.

Project 9 | **AI-Powered Personal Finance Tracker & Expense Classifier**

Category: Machine Learning + NLP + AI Web App | Duration: 5–6 weeks

Abstract

A Streamlit web application that automatically categorizes personal financial transactions using a Naive Bayes text classifier. Users input or upload bank statement CSVs and the system categorizes expenses (food, transport, utilities, etc.), forecasts next-month spending using Linear Regression, and generates budget alerts. All data stored in MySQL locally.

Real-World Problem Statement

Most people in Pakistan do not have access to smart banking tools. This offline-capable AI assistant empowers individuals to understand and control their spending without sharing data with any cloud service.

Functional Requirements

- Manual transaction entry or CSV bank statement upload
- ML model auto-categorizes each transaction into 10+ categories
- Monthly spending breakdown with interactive Plotly charts
- Budget setting per category with threshold alerts
- Next-month expense forecast using Linear Regression per category
- MySQL storage for all transactions, budgets, and forecasts
- Export full monthly financial report as PDF

Tools & Technologies

Python 3.x • Streamlit • Scikit-learn (Naive Bayes, Linear Regression, TF-IDF) • Pandas • Plotly • MySQL • ReportLab (PDF export) • OpenPyXL

Prize Potential: FinTech + AI is a globally funded vertical — this project wins in entrepreneurship and startup-track competitions.

Project 10 | Multi-Class Human Emotion Recognition from Facial Expressions

Category: Computer Vision + Deep Learning + AI Web App | Duration: 7–8 weeks

Abstract

A real-time facial emotion recognition system using a PyTorch CNN trained on the FER-2013 dataset (Kaggle, free). The system detects faces via OpenCV Haar Cascade and classifies emotions into 7 categories (angry, disgust, fear, happy, neutral, sad, surprise) from webcam feed or uploaded images. Results are displayed live on a Streamlit dashboard with session emotion analytics stored in MySQL.

Real-World Problem Statement

Emotion recognition has applications in mental health monitoring, online exam proctoring, and human-computer interaction. This project demonstrates a complete AI pipeline from raw image to real-world decision-making.

Functional Requirements

- Live webcam feed or image upload for emotion detection
- Detects and tracks multiple faces simultaneously using OpenCV
- Classifies each detected face into 7 emotion categories in real-time
- Confidence bar chart shown for each detection
- Session summary: dominant emotion over session, emotion timeline graph
- MySQL logs all detections with timestamps for later analysis
- Admin analytics: emotion distribution across all sessions

Tools & Technologies

Python 3.x • PyTorch + torchvision • OpenCV (Haar Cascade face detection) • Streamlit • Matplotlib / Plotly • MySQL • Pillow • Kaggle FER-2013 Dataset (free, 35,887 images)

Prize Potential: Emotion AI + live demo capability = guaranteed crowd engagement at any tech expo or national competition demo round.

Evaluation Criteria Alignment

Each project is evaluated against the following criteria aligned with CS-III course objectives:

Criterion	Description
Problem Relevance (15%)	Does the project address a real industry or societal problem?
Technical Implementation (25%)	Correct use of AI algorithms (ML, NLP, Computer Vision, etc.)
Code Quality (15%)	Modular, documented, well-structured Python code
UI/UX Design (15%)	Streamlit interface is intuitive, functional, and presentable
Database Integration (10%)	MySQL properly stores, retrieves, and manages application data
Results & Accuracy (10%)	Model performance metrics reported (accuracy, F1, confusion matrix)
Presentation & Demo (10%)	Live demo works flawlessly on local machine during viva